JASP Student Developers

Unit Testing for JASP Modules

13.05.2026

julian.wuth@student.uva.nl

Contents

Contents

Intro to unit testing

Why unit testing?

1. Ensures that your code behaves as expected
2. Fewer bugs
3. Alerts you to unwanted changes later
4. Shows you where your code might need restructuring
5. Implement changes without worrying about breaking your code (robust code)

Unit testing in JASP

Prerequisites

- Having jaspTools installed
- JASP tools enables you to ...:
- ... run JASP analyses from R
- ... test JASP analyses

Unit testing in JASP

Folder structure

- ModuleName/
    - tests/
        - testthat/
            - _snaps/
            - (jaspfiles)
        - testthat.R
    - .github/workflows/unittests.yml

Unit testing in JASP

Creating a test file

- Located in *tests/testthat/*
- Naming convention is test-“analysisname”.R
- Every analysis has a separate test file

Unit testing in JASP

Tests for tables

- Use `jaspTools::expect_equal_tables()` inside `testthat()`
- `expect_equal_tables()` has 3 arguments:
  1. test: The new table output to be tested
  2. ref: Table snapshot to be compared to. Generated with `jaspTools::makeTestTable()`
  3. label: Label for the expectation

Unit testing in JASP

Tests for tables

```
options <- jaspTools::analysisOptions("BinomialTest")
options$variables <- "contBinom"
options$testValue <- 0.5
results <- jaspTools::runAnalysis("BinomialTest", "test.csv", options)
table <- results[["results"]][["binomialTable"]][["data"]]
jaspTools::makeTestTable(table)
```

```
test_that("Binomial table results match", {
  options <- jaspTools::analysisOptions("BinomialTest")
  options$variables <- "contBinom"
  options$testValue <- 0.5
  results <- jaspTools::runAnalysis("BinomialTest", "test.csv", options)
  table <- results[["results"]][["binomialTable"]][["data"]]
  jaspTools::expect_equal_tables(table,
         list("TRUE", 58, 0, 0.133210619207213, 0.58, 100, "contBinom", "FALSE",
              42, 1, 0.133210619207213, 0.42, 100, "contBinom"))
})
```

Unit testing in JASP

Tests for plots

- Use `jaspTools::expect_equal_plots()` inside `testthat()`
- `expect_equal_plots()` has 2 arguments:
  1. test: The new plot to be tested
  2. name: Name the plot is/will be stored under in the snapshot folder
- Creates an .svg file when the test is run for the first time using `jaspTools::testAll()` or `jaspTools::testAnalysis()`
- This file serves as a reference for future tests

Unit testing in JASP

Tests for plots

```
test_that("Descriptives plot matches", {
  options <- jaspTools::analysisOptions("BinomialTest")
  options$variables <- "contBinom"
  options$testValue <- 0.5
  options$descriptivesPlots <- TRUE
  results <- jaspTools::runAnalysis("BinomialTest", "test.csv", options)
  plotName <- results[["results"]][["containerPlots"]][["collection"]][["containerPlots_contBinom"]][["collection"]]
  testPlot <- results[["state"]][["figures"]][[plotName]][["obj"]]
  expect_equal_plots(testPlot, "descriptives")
})
```

```
jaspTools::testAnalysis("BinomialTest")
```

Unit testing in JASP

Testing error handling

```
test_that("Analysis handles errors", {
  options <- jaspTools::analysisOptions("RegressionLinear")

  options$dependent <- "debInf"
  options$covariates <- "contGamma"
  options$modelTerms <- list(list(components="contGamma", isNuisance=FALSE))
  results <- jaspTools::runAnalysis("RegressionLinear", "test.csv", options)
  expect_identical(results[["status"]], "validationError", label = "Inf dependent check")

  options$dependent <- "contNormal"
  options$covariates <- "debInf"
  options$modelTerms <- list(list(components="debInf", isNuisance=FALSE))
  results <- jaspTools::runAnalysis("RegressionLinear", "test.csv", options)
  expect_identical(results[["status"]], "validationError", label = "Inf covariate check")
})
```

Unit testing in JASP

Creating tests automatically

1. Using `jaspTools::runAnalysis()` with `makeTests = TRUE`

```
options <- jaspTools::analysisOptions("BinomialTest")
options$variables <- "contBinom"
options$testValue <- 0.5
options$descriptivesPlots <- TRUE
results <- jaspTools::runAnalysis("BinomialTest", "test.csv", options, makeTests = TRUE)
```

Unit testing in JASP

Creating tests automatically

2. Auto-generate from JASP file(s)

- Move JASP files to one of:
  - *tests/testthat/jaspfiles/library/*
  - *tests/testthat/jaspfiles/verified/*
  - *tests/testthat/jaspfiles/other/*

Unit testing in JASP

Creating tests automatically

- Set up jaspTools and run `jaspTools::makeTestsFromExamples()`

```
 # to update to the current version of jaspTools
renv::install("C:/JASP-Packages/jaspTools")
library(jaspTools)
setupJaspTools(pathJaspDesktop = "XXX", installJaspModules = FALSE, installJaspCorePkgs = FALSE,
quiet = FALSE, force = TRUE)

# to install the latest version of the module
renv::install(".")
setPkgOption("module.dirs", ".")
setPkgOption("reinstall.modules", FALSE)

# to generate unit test based on the files in the `examples` folder
makeTestsFromExamples()
```

Unit testing in JASP

Creating tests automatically

- Finally, run `jaspTools::testAll()` to check that all tests pass

Contents

Live-Walkthrough

Creating unit tests for JASP modules

- Let’s see how this works in practice
- Clone this repo to follow along

See you next time!